



PES

UNIVERSITY

CELEBRATING 50 YEARS

PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

PROGRAMMING WITH PYTHON

Object-Oriented Programming (OOP) and Advanced Concepts

Classes and Objects

Lekha A

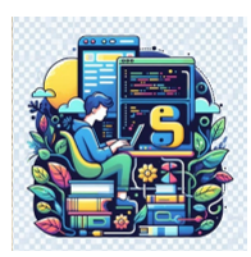
Computer Applications



PROGRAMMING WITH PYTHON

Classes

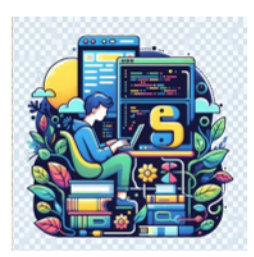
- Classes and objects are fundamental concepts used to model and represent real-world entities, data, and behavior in a program.
- These concepts help organize code, promote encapsulation, and enable code reusability.



PROGRAMMING WITH PYTHON

Classes

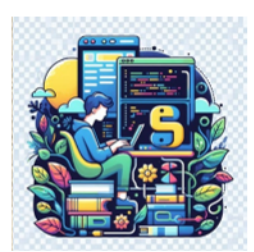
- Classes are blueprints for creating objects.
 - They define the structure and behavior of objects, including their properties and methods.
 - Class can be considered as a general category or a prototype for objects.



PROGRAMMING WITH PYTHON

Classes

- For example
 - Have a Laptop class that defines attributes like brand, model and price, and methods like `turn_on()`, `turn_off()`, and `display_info()`.
- Classes can be seen as user-defined data types that encapsulate data and related operations.



PROGRAMMING WITH PYTHON

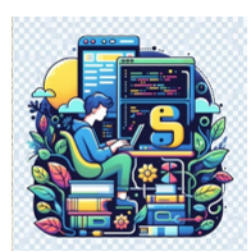
Classes

- Example

```
1
class Laptop:
    2
    def __init__(self, brand, model, price):
        self.brand = brand
        self.model = model
        self.price = price

    def turn_on(self):
        print(f"{self.brand} {self.model} is now ON.")

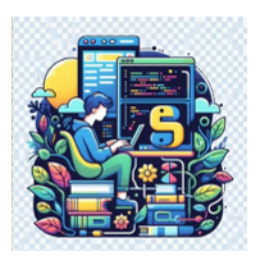
    def turn_off(self):
        print(f"{self.brand} {self.model} is now OFF.")
```



PROGRAMMING WITH PYTHON

Classes

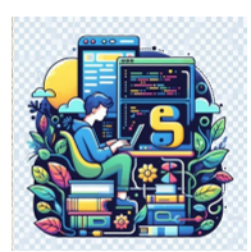
- At **1** we define a class called Laptop.
- By convention, capitalized names refer to classes in Python.
- The `__init__()` Method is a function that's part of a class is a method.
- The `__init__()` method at **2** is a special method Python runs automatically whenever we create a new instance based on the Laptop class.



PROGRAMMING WITH PYTHON

Classes

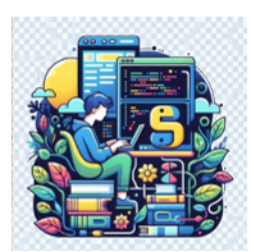
- This method has two leading underscores and two trailing underscores, a convention that helps prevent Python's default method names from conflicting with user method names.
- The `__init__()` method is defined to have four parameters: `self`, `brand`, `model` and `price`.



PROGRAMMING WITH PYTHON

Classes

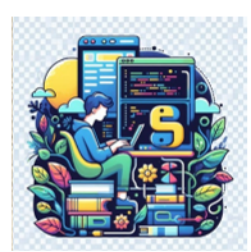
- The self parameter is required in the method definition, and it must come first before the other parameters.
- It must be included in the definition because when Python calls this `__init__()` method, the method call will automatically pass the self argument.



PROGRAMMING WITH PYTHON

Classes

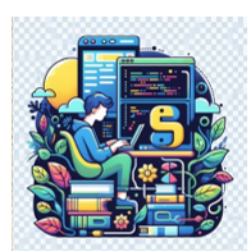
- Every method call associated with a class automatically passes self, which is a reference to the instance itself; it gives the individual instance access to the attributes and methods in the class.
- When an instance of Laptop is made, Python will call the `__init__()` method from the Laptop class.



PROGRAMMING WITH PYTHON

Classes

- The user will pass Laptop() brand, model and price as arguments; self is passed automatically, so user does not need to pass it.
- Whenever an instance is made from the Laptop class, user needs to provide values for only the last three parameters: brand, model and price.



PROGRAMMING WITH PYTHON

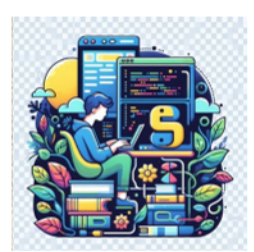
Classes

- Example

```
class Laptop:
    def __init__(self, brand, model, price):
3       self.brand = brand
        self.model = model
        self.price = price

    def turn_on(self):
        print(f"{self.brand} {self.model} is now ON.")

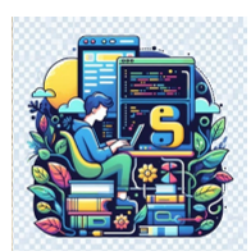
    def turn_off(self):
        print(f"{self.brand} {self.model} is now OFF.")
```



PROGRAMMING WITH PYTHON

Classes

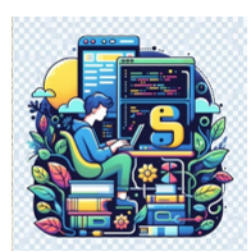
- The two variables defined at **3** each have the prefix self.
 - Any variable prefixed with self is available to every method in the class, and user will also be able to access these variables through any instance created from the class.
 - `self.brand = brand` takes the value stored in the parameter brand and stores it in the variable brand, which is then attached to the instance being created.



PROGRAMMING WITH PYTHON

Classes

- The same process happens with `self.model = model`, `self.price=price`.
- Variables that are accessible through instances like this are called attributes.



PROGRAMMING WITH PYTHON

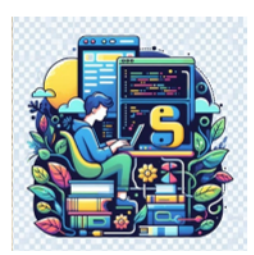
Classes

- Example

```
class Laptop:
    def __init__(self, brand, model, price):
        self.brand = brand
        self.model = model
        self.price = price

    4 def turn_on(self):
        print(f"{self.brand} {self.model} is now ON.")

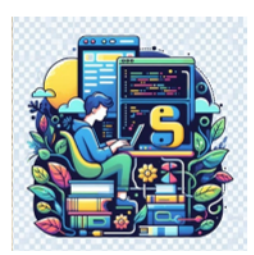
    def turn_off(self):
        print(f"{self.brand} {self.model} is now OFF.")
```



PROGRAMMING WITH PYTHON

Classes

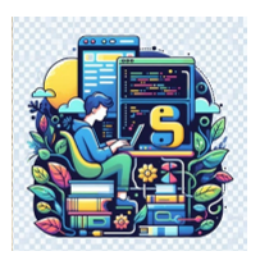
- The Laptop class has another method defined: `turn_on()` at **4**.
- Because these methods don't need additional information like a brand or model they are defined to have one parameter, `self`.



PROGRAMMING WITH PYTHON

Objects

- Objects are instances of classes.
 - They contain the data and behavior that are defined by the class.
 - It is a concrete representation of the class's blueprint, with its own unique values for attributes and the ability to invoke methods defined in the class.



PROGRAMMING WITH PYTHON

Objects

- Objects are instances of classes.
 - Objects are created based on the class definition.
 - Each object has its own set of attributes and can perform actions using the methods defined in the class.
 - Making an object from a class is called **instantiation**.



PROGRAMMING WITH PYTHON

Objects

- Example

1

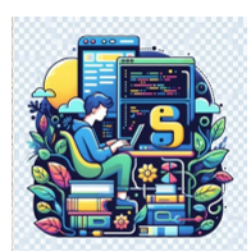
```
laptop1 = Laptop("Dell", "Inspiron 15", 55000)  
laptop2 = Laptop("Apple", "MacBook Air", 120000)
```



PROGRAMMING WITH PYTHON

Objects

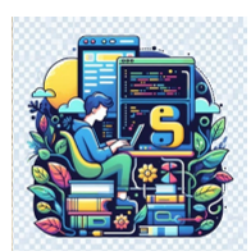
- At **1** user tells Python to create a Laptop whose 'brand' is Dell, 'model' is Inspiron and price is 55000.
- When Python reads this line, it calls the `__init__()` method in Laptop with the arguments "Dell", "Inspiron", 55000.



PROGRAMMING WITH PYTHON

Objects

- The `__init__()` method creates an instance representing this particular car/dog and sets the name and age attributes using the values provided.
- The `__init__()` method has no explicit return statement, but Python automatically returns an instance representing this car/dog.
- Store that instance in the variable `laptop1`.



PROGRAMMING WITH PYTHON

Recap

- Classes
- Objects



THANK YOU

Lekha A
Computer Applications